

6.1 Введение в ООП

6.1.1 Что такое объектно-ориентированное программирование?

Объектно-ориентированное программирование, или ООП - это подход к написанию программ, в котором программа строится вокруг **объектов**.

Объект в такой модели объединяет **данные** о себе и **действия**, которые он умеет выполнять. В Java это выражается через **классы, объекты, поля и методы**.

Проще говоря, в ООП мы думаем не только в стиле «что сделать», но и в стиле «кто это делает и что у него внутри»

6.1.2 Зачем вообще нужен ООП и какие задачи он решает?

ООП нужно для того, чтобы программу было проще **проектировать, читать, изменять и расширять**.

Когда код разбит на отдельные объекты, каждый объект отвечает за свою часть логики. Это помогает не смешивать всё в одном месте.

Такой подход решает несколько важных задач:

1. Упрощает структуру программы.

Большую программу легче собирать из отдельных частей, чем писать как один сплошной набор команд.

2. Прячет внутреннюю реализацию.

Снаружи мы используем объект через его методы, не влезая в его внутреннее устройство.

3. Повышает повторное использование кода.

Если объект или класс уже написан, его можно использовать в другой части программы или в другом проекте.

4. Упрощает поддержку и замену частей программы.

Если один объект работает плохо, его проще исправить или заменить, не переписывая всё приложение.

6.1.3 Чем ООП отличается от процедурного подхода?

В **процедурном подходе** программа обычно строится вокруг **последовательности действий**, функций и шагов обработки данных.

В **ООП** программа строится вокруг **объектов**, которые объединяют в себе и данные, и методы для работы с этими данными.

То есть разница в том, **на чём делается основной акцент**:

процедурный подход - что сделать;

ООП - с кем мы работаем, что у этого объекта есть и что он умеет.

Например, в процедурном стиле можно отдельно хранить данные о машине и отдельно писать функции для работы с ними.

В ООП логичнее сделать объект Car, у которого есть состояние и методы, например start(), stop(), accelerate().

Так код обычно получается ближе к реальным сущностям предметной области и легче воспринимается.

6.1.4 Что такое объект простыми словами?

Объект - это конкретная сущность, с которой работает программа.

Если говорить совсем просто, объект - это **что-то созданное по шаблону класса**, у чего есть свои данные и свои действия.

Например:

Кот, БанковскаяКарта, Пользователь, Заказ - всё это можно представить как объекты.

У объекта есть:

состояние - какие у него сейчас данные;

поведение - что он умеет делать.

Например, объект Кот может иметь состояние: имя, возраст, цвет.

И поведение: спать, мяукать, есть.

В Java объект - это **экземпляр класса**, созданный во время работы программы.

6.1.5 Что такое класс простыми словами?

Класс - это **описание, шаблон** или **чертёж**, по которому создаются объекты.

Он задаёт, какие данные будут у объекта и какие действия он сможет выполнять.

Например, класс Car может описывать:

- поля: brand, speed, color
- методы: start(), stop(), accelerate()

Сам класс - это ещё не конкретная машина.

Это только описание того, **какими будут все объекты этого типа**.

В Java класс объявляется через ключевое слово class.

6.1.6 Чем класс отличается от объекта?

Класс - это **шаблон**.

Объект - это **конкретный экземпляр**, созданный по этому шаблону.

Например, класс Dog - это общее описание собаки.

А объект dog1 - это уже конкретная собака, созданная в программе.

Можно представить так:

класс - это чертёж дома;

объект - это уже построенный дом.

По одному классу можно создать **много объектов**.

При этом у всех объектов будет общая структура, но значения полей могут отличаться.

Например, у двух объектов класса Dog могут быть разные клички и возраст.

6.1.7 Что такое состояние и поведение объекта?

Состояние объекта - это его текущие данные.

Поведение объекта - это действия, которые он умеет выполнять.

В Java состояние объекта обычно хранится в **полях**, а поведение описывается через **методы**.

Например, для объекта Bicycle:

состояние: скорость, передача, частота педалирования;

поведение: переключить передачу, увеличить скорость, затормозить.

Это одна из самых важных идей ООП: объект не просто хранит данные, а ещё и умеет с ними работать.

6.1.8 Что такое поля и методы объекта?

Поля - это переменные внутри объекта, которые хранят его состояние.

Их ещё называют **атрибутами** или **свойствами** объекта.

Методы - это действия, которые объект умеет выполнять.

Методы работают с данными объекта и описывают его поведение.

Пример:

```
class Car {  
    String brand; // поле  
    int speed;    // поле  
  
    void start() { // метод  
        System.out.println("Машина заведена");  
    }  
  
    void accelerate() { // метод  
        speed++;  
    }  
}
```

Здесь brand и speed - это поля, а start() и accelerate() - методы.

Важно понимать:

поля отвечают за то, что объект хранит,
методы отвечают за то, что объект делает.

6.1.9 Как создать объект в Java? Что делает оператор new?

Чтобы создать объект в Java, обычно пишут имя класса, затем оператор **new**, затем вызывают **конструктор**.

Пример:

```
Car car = new Car();
```

Что здесь происходит:

1. Car - это класс.
2. new Car() - создание нового объекта этого класса.
3. car - переменная, которая будет хранить ссылку на созданный объект.

Оператор **new** делает две важные вещи:

1. создаёт новый объект;
2. возвращает **ссылку** на этот объект.

Также при создании объекта вызывается **конструктор** - специальный механизм инициализации нового объекта.

Важно:

если просто написать переменную ссылочного типа, объект ещё **не создаётся**.
Объект появляется только после использования new.

6.1.10 Что такое ссылка на объект?

Ссылка на объект - это значение, через которое программа может обратиться к объекту.

Это **не сам объект**, а способ получить к нему доступ.

Когда мы пишем:

```
Car car = new Car();
```

переменная `car` хранит не весь объект целиком, а **ссылку** на него.

Через эту ссылку можно обращаться к полям и методам объекта:

```
car.start();  
car.speed = 10;
```

Важно понимать несколько вещей:

1. Одна ссылка - это не объект.

Это только переменная, которая указывает на объект.

2. Несколько ссылок могут указывать на один и тот же объект.

То есть один объект может быть доступен из разных переменных.

3. Ссылка может быть `null`.

Это означает, что сейчас она ни на какой объект не ссылается.

Если у программы больше не осталось ссылок на объект, такой объект со временем может быть удалён сборщиком мусора.

6.2 Классы и объекты

6.2.1 Как описывается класс в Java?

Класс в Java описывается через объявление с ключевым словом **class**. У класса есть **имя** и **тело класса** в фигурных скобках { }. Внутри этого тела обычно размещают поля, методы и конструкторы. Перед именем класса могут стоять модификаторы доступа, например `public`.

Пример:

```
public class Car {  
}
```

6.2.2 Что можно хранить внутри класса?

Внутри класса можно хранить **поля**, **методы** и **конструкторы**. Кроме этого, в Java внутри класса могут быть вложенные классы, интерфейсы и блоки инициализации, но для начала важно запомнить главное: класс обычно описывает **данные объекта** и **действия объекта**

6.2.3 Что такое экземпляр класса?

Экземпляр класса - это конкретный объект, созданный по этому классу. Класс - это описание, а экземпляр - уже реальный объект в памяти, с которым работает программа. В Java объект создаётся динамически, а переменная ссылочного типа хранит ссылку на него.

6.2.4 Что такое поля экземпляра?

Поля экземпляра - это поля без ключевого слова **static**. Они принадлежат не классу целиком, а **каждому отдельному объекту**. Это значит, что у каждого объекта есть **своя собственная копия** таких полей. Например, у двух объектов `Car` поле `speed` будет одно по имени, но значение у каждого объекта может быть своим.

6.2.5 Что такое методы экземпляра?

Методы экземпляра - это методы без ключевого слова **static**. Они вызываются у конкретного объекта и обычно работают с его полями. То есть такой метод выполняет действие именно для того объекта, у которого он был вызван.

6.2.6 Как обращаться к полям и методам объекта?

К полям и методам объекта обращаются через **ссылку на объект** и **точку**. Для поля форма такая: `objectReference.fieldName`, а для метода - `objectReference.methodName()`. Это стандартный способ работы с объектами в Java.

Пример:

```
Car car = new Car();  
car.speed = 10;  
car.start();
```

6.2.7 Что такое **this** и зачем он нужен?

this - это ссылка на **текущий объект**, внутри которого сейчас выполняется код. Он нужен, когда надо явно показать: речь идёт именно о поле или методе текущего объекта. Чаще всего **this** используют, когда имя параметра совпадает с именем поля. Тогда `this.field` означает поле объекта, а просто `field` может означать параметр или локальную переменную.

Пример:

```
public class Point {  
    int x;  
    int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

6.2.8 Можно ли внутри класса обращаться к своим полям и методам без **this**?

Да, **можно**. Если внутри метода или конструктора нет неоднозначности, Java и без **this** понимает, что речь идёт о поле или методе текущего объекта. Поэтому **this** часто опускают, когда он не нужен для ясности.

Пример:

```
public void printInfo() {  
    System.out.println(speed);  
    start();  
}
```

6.2.9 Когда **this** обязателен, а когда нет?

this **обязателен**, когда без него возникает путаница. Например, если параметр конструктора называется так же, как поле объекта: `this.name = name;`.

Ещё **this** используется для вызова другого конструктора того же класса через **this()**.

Во всех обычных случаях, когда имена не конфликтуют, обращаться к своим полям и методам можно и без **this**.

6.3 Конструкторы

6.3.1 Что такое конструктор?

Конструктор - это специальная часть класса, которая вызывается при создании нового объекта. Его задача - **инициализировать объект**, то есть задать ему начальное состояние. В объявлении конструктора используется **имя класса**, и у него **нет типа возвращаемого значения**.

Пример:

```
public class Car {  
    String brand;  
  
    public Car(String brand) {  
        this.brand = brand;  
    }  
}
```

6.3.2 Чем конструктор отличается от обычного метода?

Конструктор отличается от метода тем, что он:

1. называется так же, как класс;
2. не имеет возвращаемого типа;
3. вызывается при создании объекта;
4. не используется как обычный метод через точку, например `obj.someMethod()`.

Обычный метод выполняет действие уже у существующего объекта, а конструктор участвует именно в его создании и начальной настройке.

6.3.3 Что делает конструктор по умолчанию?

Если в классе **вообще не объявлен ни один конструктор**, компилятор сам добавляет **конструктор по умолчанию** без параметров. Такой конструктор вызывает конструктор родительского класса без параметров. Если у родителя такого конструктора нет, программа не скомпилируется. Поля объекта при создании всё равно получают свои стандартные начальные значения, если им не задано другое значение явно.

6.3.4 Что такое конструктор без параметров и конструктор с параметрами?

Конструктор без параметров - это конструктор, который вызывается без аргументов. Он обычно задаёт объекту какие-то стандартные начальные значения.

Конструктор с параметрами принимает значения снаружи и позволяет сразу создать объект в нужном состоянии.

Пример:

```
public class Car {  
    String brand;  
    int speed;  
  
    public Car() {  
        brand = "Unknown";  
        speed = 0;  
    }  
  
    public Car(String brand, int speed) {  
        this.brand = brand;  
        this.speed = speed;  
    }  
}
```

6.3.5 Можно ли перегружать конструкторы?

Да, **конструкторы можно перегружать**. Это значит, что в одном классе может быть несколько конструкторов, если у них **разные списки параметров**. Java различает их по количеству параметров и по их типам. А вот два конструктора с полностью одинаковой сигнатурой создать нельзя.

6.3.6 Что такое вызов `this()` внутри конструктора?

Вызов `this()` внутри конструктора - это обращение к **другому конструктору этого же класса**. Такой приём используют, чтобы не дублировать одинаковую логику инициализации.

Важно: если `this()` используется, он должен стоять **первой строкой** конструктора.

Пример:

```
public class Rectangle {  
    int width;  
    int height;  
  
    public Rectangle() {  
        this(1, 1);  
    }  
  
    public Rectangle(int width, int height) {  
        this.width = width;  
        this.height = height;  
    }  
}
```

6.3.7 Зачем инициализировать объект через конструктор? Где можно не делать это через конструктор?

Инициализация через конструктор нужна затем, чтобы объект **сразу после создания** был в корректном и понятном состоянии. Это особенно важно для обязательных данных, без которых объект не должен существовать. Конструктор удобен ещё и тем, что внутри него можно использовать логику и проверки, а не только простое присваивание.

Плюс! Очень важно! Внедрение через конструктор – позволяет легко заменить реализацию поля, в которое происходит внедрение

6.4 Основные принципы ООП

6.4.1 Какие основные принципы ООП существуют?

Обычно в ООП выделяют **четыре основных принципа: инкапсуляцию, наследование, полиморфизм и абстракцию**. В Java они проявляются через классы, объекты, модификаторы доступа, наследование, переопределение методов, а также через `abstract`-классы и интерфейсы.

6.4.2 Что такое инкапсуляция как принцип ООП?

Инкапсуляция - это объединение данных и методов работы с ними внутри объекта, плюс сокрытие внутреннего состояния от прямого внешнего доступа. Проще говоря, объект сам контролирует, как с ним можно работать.

6.4.3 Что такое наследование?

Наследование - это механизм, при котором один класс может быть построен на основе другого и получать его поля и методы. Это позволяет переиспользовать уже существующий код и расширять поведение базового класса.

6.4.4 Что такое полиморфизм?

Полиморфизм - это возможность работать с объектами разных подклассов через общий тип. При этом Java вызывает метод не по типу переменной, а по реальному типу объекта, который в ней лежит.

6.4.5 Что такое абстракция?

Абстракция - это выделение главного и сокрытие лишних деталей реализации. В Java для этого часто используют **абстрактные классы и интерфейсы**: они задают общее поведение, а конкретные классы уже реализуют детали.

6.4.6 Как эти принципы работают вместе в реальном коде?

Обычно это выглядит так: базовый тип задаёт **общую идею** объекта, наследники **уточняют поведение**, поля прячут через **инкапсуляцию**, а код работает с объектами через **общий тип**, используя **полиморфизм**. За счёт этого код получается более понятным, расширяемым и безопасным для изменений.

Пример:

```
abstract class Payment {
    private int amount;
    public Payment(int amount) {
        if (amount <= 0) {throw new IllegalArgumentException("Сумма должна быть больше нуля");}
        this.amount = amount;
    }
    public int getAmount() {
        return amount;
    }
    public abstract void process();
}

class CardPayment extends Payment {
    public CardPayment(int amount) {super(amount);}
    @Override
    public void process() {
        System.out.println("Оплата картой: " + getAmount());
    }
}

class SbpPayment extends Payment {
    public SbpPayment(int amount) {super(amount);}

    @Override
    public void process() {
        System.out.println("Оплата по СБП: " + getAmount());
    }
}
```

Здесь **инкапсуляция** - это `private amount`, **абстракция** - `abstract class Payment`, **наследование** - `CardPayment` и `SbpPayment`, **полиморфизм** - возможность хранить оба объекта в переменной типа `Payment` и вызывать `process()`.

6.5 Инкапсуляция

6.5.1 Что такое инкапсуляция?

Инкапсуляция - это принцип, при котором объект хранит своё состояние внутри себя и предоставляет доступ к нему через методы. Внешний код не должен напрямую ломать внутреннюю логику объекта.

6.5.2 Зачем скрывать внутреннее состояние объекта?

Скрывать внутреннее состояние нужно затем, чтобы объект сам проверял, какие изменения допустимы, а какие нет. Тогда внешний код не сможет случайно записать в объект неправильные данные и сломать его состояние.

6.5.3 Что такое геттеры и сеттеры и зачем они нужны?

Геттер - это метод для чтения значения поля. **Сеттер** - это метод для изменения значения поля. Они нужны не просто “для красоты”, а чтобы контролировать доступ к данным: например, разрешить чтение, но запретить изменение, или добавить проверку перед изменением значения.

6.5.4 Почему не стоит делать все поля `public`?

Если сделать поля `public`, любой код сможет менять их напрямую, без проверок. Это связывает программу с конкретной внутренней реализацией класса и делает код менее гибким: изменить устройство класса потом становится сложнее. В Java прямо рекомендуют избегать `public`-полей, кроме констант.

6.5.5 Какие модификаторы доступа есть в Java?

В Java есть `private`, `protected`, `public` и доступ **без модификатора**, который называют **`package-private`**. Для верхнеуровневых классов доступны только `public` и `package-private`, а для полей и методов - все четыре варианта.

6.5.6 Чем отличаются `private`, `package-private`, `protected` и `public`?

`private` - доступ только внутри самого класса. **`package-private`** - доступ только внутри того же пакета. **`protected`** - доступ внутри пакета и ещё из подклассов в других пакетах. **`public`** - доступ отовсюду.

6.4.7 Когда сеттер лучше не делать вообще?

Сеттер лучше не делать, когда значение **не должно меняться после создания объекта**, либо когда менять его можно только по специальным правилам. В таких случаях поле делают только для чтения или изменяют через более осмысленные методы, а не через общий `set . . . ( )`.

6.5.8 Как инкапсуляция помогает защищать объект от некорректного состояния?

Инкапсуляция позволяет не пускать внешний код напрямую к полям, а пропускать изменения только через методы с проверками. Например, если у велосипеда только 6 передач, метод может отклонить попытку установить передачу 0 или 100, и объект останется корректным.

6.6 Наследование

6.6.1 Что такое наследование в Java?

Наследование в Java - это создание нового класса на основе уже существующего. Новый класс получает члены родительского класса и может добавлять собственные поля и методы или переопределять унаследованные методы.

6.6.2 Зачем одному классу наследоваться от другого?

Наследование нужно, когда новый класс действительно является **разновидностью** уже существующего класса и должен переиспользовать его поведение. Это уменьшает дублирование кода и позволяет расширять уже готовую логику.

6.6.3 Что такое родительский и дочерний класс?

Родительский класс - это класс, от которого наследуются. Его ещё называют **superclass**. **Дочерний класс** - это класс, который наследуется от родителя. Его ещё называют **subclass**.

6.6.4 Что наследуется от родительского класса, а что нет?

Подкласс наследует поля, методы и вложенные классы родителя, но **конструкторы не наследуются**, потому что они не являются обычными членами класса. Также подкласс **не наследует private-члены** напрямую. Доступ к остальным членам зависит от модификатора доступа и от того, в каком пакете находится подкласс.

6.6.5 Что такое extends?

extends - это ключевое слово, которым в Java указывают, что один класс наследуется от другого. Оно ставится в объявлении класса.

Пример:

```
class Animal {  
}
```

```
class Dog extends Animal {  
}
```

6.6.6 Можно ли в Java наследоваться от нескольких классов?

Нет, в Java класс может иметь только **один прямой родительский класс**. То есть множественного наследования классов в Java нет.

6.6.7 Что такое super и зачем он нужен?

super - это ссылка на родительскую часть текущего объекта. Через **super** можно

обратиться к методу или полю родительского класса, если в дочернем классе оно было переопределено или скрыто.

6.6.8 Как вызывать конструктор родительского класса?

Конструктор родительского класса вызывают через **super ()** или **super (параметры)**. Такой вызов должен стоять **первой строкой** конструктора дочернего класса. Если его не написать явно, компилятор попытается сам вызвать конструктор родителя без параметров.

6.6.9 Когда наследование полезно, а когда оно только вредит?

Наследование полезно, когда между классами есть нормальная связь вида **“один класс является более конкретной версией другого”**. Оно начинает вредить, когда его используют просто ради экономии кода: тогда дочерний класс слишком сильно зависит от внутреннего устройства родителя, и систему становится сложнее менять и поддерживать.

6.7 Полиморфизм

6.7.1 Что такое полиморфизм простыми словами?

Полиморфизм - это возможность работать с разными объектами через **один общий тип** и вызывать у них один и тот же метод, получая разное поведение. Проще говоря, снаружи вызов выглядит одинаково, но конкретный результат зависит от того, объект какого класса реально лежит в ссылке.

6.7.2 Зачем нужен полиморфизм в Java?

Он нужен, чтобы код был **гибче** и **расширяемоее**. Можно писать логику не под каждый конкретный класс отдельно, а под общий родительский класс или интерфейс, а Java сама вызовет нужную реализацию метода во время выполнения.

6.7.3 Что такое статический полиморфизм?

В учебном контексте **статическим полиморфизмом** обычно называют **перегрузку методов**. Здесь выбор нужного метода происходит **во время компиляции**, на основе имени метода, количества и типов аргументов.

6.7.4 Что такое перегрузка методов (over loading)?

Перегрузка - это когда в одном классе есть несколько методов с **одним и тем же именем**, но с **разными параметрами**. Разными должны быть именно параметры: их количество, порядок или типы.

6.7.5 По чему Java выбирает перегруженный метод?

Java выбирает перегруженный метод по **списку аргументов**: смотрит, какие методы подходят, и выбирает **наиболее подходящий и наиболее специфичный**. При этом **возвращаемый тип** сам по себе в выборе не участвует.

6.7.6 Что такое динамический полиморфизм?

Динамический полиморфизм связан с **переопределением методов**. Здесь метод, который реально будет вызван, определяется **во время выполнения программы** по фактическому типу объекта, а не только по типу переменной.

6.7.7 Что такое переопределение методов (overriding)?

Переопределение - это когда дочерний класс заново задаёт реализацию метода, который уже есть у родительского класса. Для этого у метода должно совпадать имя и параметры, а возвращаемый тип должен совпадать или быть более узким совместимым типом.

6.7.8 Как Java определяет, какой переопределенный метод вызвать во время выполнения?

Для **обычных методов объекта** Java смотрит на **реальный класс объекта в памяти**. Поэтому если переменная типа `Animal` хранит объект `Cat`, то при вызове переопределённого метода будет выполнена версия из `Cat`.

6.7.9 Чем перегрузка отличается от переопределения?

Перегрузка - это несколько методов с одним именем, но разными параметрами, и выбор идёт **на этапе компиляции**. **Переопределение** - это новая реализация уже существующего метода в дочернем классе, и выбор нужной реализации идёт **во время выполнения**.

6.7.10 Что такое раннее и позднее связывание?

В упрощённом учебном смысле **раннее связывание** - это когда выбор можно сделать **во время компиляции**, например при перегрузке. **Позднее связывание** - это когда окончательный выбор делается **во время выполнения**, как при вызове переопределённого метода объекта.

6.7.11 Почему переменная типа родителя может ссылаться на объект дочернего класса?

Потому что объект дочернего класса одновременно является и объектом своего родительского типа. В Java такое присваивание допустимо и происходит как **неявное приведение вверх**, потому что дочерний класс расширяет родительский.

6.7.12 Что можно вызвать через ссылку родительского типа, а что нельзя?

Через ссылку родительского типа можно обращаться только к тем полям и методам, которые **видны через тип родителя**. Если метод переопределён, выполнится версия дочернего класса, но члены, которые есть только у дочернего класса, без приведения вызвать нельзя. Для полей важно помнить, что доступ к ним определяется **типом ссылки**, а не фактическим типом объекта.

6.7.13 Что такое `upcasting` и `downcasting`?

Upcasting - это приведение от дочернего типа к родительскому; обычно оно происходит автоматически и безопасно. **Downcasting** - это приведение от родительского типа к дочернему; оно пишется явно и во время выполнения Java проверяет, действительно ли объект имеет нужный тип. Если нет, будет ошибка приведения.

6.7.14 Зачем нужен instanceof?

`instanceof` нужен, чтобы проверить, относится ли объект к определённому классу, подклассу или интерфейсу. Его часто используют перед `downcasting`, чтобы не получить ошибку во время выполнения. Ещё важно помнить, что `null` не является экземпляром никакого типа.

6.8 Абстракция, абстрактные классы и интерфейсы

6.8.1 Что такое абстракция в ООП?

Абстракция - это выделение главного в объекте и сокрытие лишних деталей реализации. В коде это означает, что мы описываем **что объект должен уметь**, не заставляя внешний код знать все внутренние детали того, **как именно** это сделано.

6.8.2 Зачем отделять “что делает” от “как делает”?

Это нужно, чтобы код был проще менять и расширять. Когда снаружи известен только контракт поведения, внутреннюю реализацию можно переделывать, не ломая весь остальной код.

6.8.3 Что такое абстрактный класс?

Абстрактный класс - это класс, который объявлен с ключевым словом **abstract**. Он может содержать и обычные методы с реализацией, и абстрактные методы без реализации, а также поля и методы с разными модификаторами доступа.

6.8.4 Можно ли создать объект абстрактного класса?

Нет, **объект абстрактного класса создать нельзя**. Такой класс нужен как основа для других классов, которые будут его расширять и завершать реализацию.

6.8.5 Что такое абстрактный метод?

Абстрактный метод - это метод, который объявлен, но не содержит реализации. Он задаёт обязательное поведение, которое должен реализовать конкретный дочерний класс, если сам этот класс не остаётся абстрактным.

6.8.6 Что такое интерфейс в Java?

Интерфейс - это ссылочный тип, который задаёт общий контракт поведения. В интерфейсе могут быть абстрактные методы, default-методы, static-методы и константы; сам интерфейс создавать через **new** нельзя, его нужно **реализовать** в классе.

6.8.7 Чем интерфейс отличается от абстрактного класса?

Абстрактный класс подходит, когда нужно дать связанным классам **общую базу**, общее состояние и часть готовой логики. Интерфейс подходит, когда нужен **общий контракт** для, возможно, вообще не связанных между собой классов; кроме того, класс может расширить только **один** класс, но реализовать **несколько интерфейсов**.

6.8.8 Когда лучше использовать интерфейс, а когда абстрактный класс?

Интерфейс обычно лучше, когда важно описать, **что объект умеет**, и дать

общий тип для разных классов. **Абстрактный класс** лучше, когда нужно ещё и поделиться общей реализацией, полями или закрытой внутренней логикой между близкими по смыслу классами.

6.8.9 Что такое `implements`?

`implements` - это ключевое слово, которым класс указывает, что он **реализует интерфейс**. Тем самым класс обязуется предоставить реализацию нужных методов интерфейса, если сам класс не абстрактный.

6.8.10 Можно ли реализовать несколько интерфейсов?

Да, в Java класс может реализовать **несколько интерфейсов сразу**. Это один из способов получить общие возможности без множественного наследования классов.

6.8.11 Что такое `default`-методы в интерфейсах?

`default`-методы - это методы интерфейса с готовой реализацией. Они появились, чтобы можно было расширять интерфейсы новыми возможностями и при этом не ломать уже существующие реализации.

6.8.12 Зачем в интерфейсах появились `static`- и `private`-методы?

`static`-методы в интерфейсах позволяют хранить служебную логику, связанную именно с этим интерфейсом, без привязки к объекту. `private`-методы появились позже, чтобы `default`- и другие неабстрактные методы интерфейса могли **делить общий внутренний код** и не дублироваться.

6.9 Переопределение, перегрузка и работа методов

6.9.1 Какие правила есть у переопределения методов?

Если очень просто, то при переопределении у метода должны совпадать **имя и параметры**. Возвращаемый тип должен совпадать или быть более узким совместимым типом, уровень доступа нельзя сделать более закрытым, а по проверяемым исключениям нельзя расширять контракт по сравнению с родительским методом.

6.9.2 Можно ли переопределить `private`, `static` и `final` методы?

private-методы не переопределяются, потому что они не наследуются. **static-методы** тоже не переопределяются - они **скрываются**. **final-методы** переопределять нельзя вообще.

6.9.3 Что делает аннотация `@Override`?

Аннотация **@Override** говорит компилятору: "я хочу переопределить метод родителя". Если окажется, что метод на самом деле не переопределяет ничего из суперкласса или интерфейса, компилятор выдаст ошибку, и это помогает раньше заметить проблему.

6.9.4 Можно ли менять возвращаемый тип при переопределении?

Да, но не как угодно. Можно вернуть **тот же тип** или **более узкий совместимый подтип** - это называется **ковариантный возвращаемый тип**.

6.9.5 Можно ли менять уровень доступа при переопределении?

Да, но только в сторону **такого же или более открытого** доступа. Например, `protected`-метод можно сделать `public`, а вот `public` превратить в `private` нельзя.

6.9.6 Что происходит, если сигнатура метода меняется совсем немного?

Если меняются параметры, то это уже не переопределение, а **другой метод**, и часто это становится **перегрузкой**. Из-за этого Java может вести себя не так, как ожидается: родительский метод не будет заменён, потому что сигнатура уже другая.

6.9.7 Почему перегрузка определяется на этапе компиляции, а переопределение - во время выполнения?

Потому что при перегрузке компилятор уже видит **имя метода и типы аргументов** и может выбрать подходящую сигнатуру заранее. А при переопределении нужно ещё понять, **объект какого класса реально лежит в ссылке во время выполнения**, и только тогда выбрать нужную реализацию метода.

6.10 Ключевые слова и особенности ООП в Java

6.10.1 Что такое static-поле и static-метод?

static-поле принадлежит **классу**, а не отдельному объекту, поэтому оно существует в одном экземпляре на весь класс. **static-метод** тоже принадлежит классу и вызывается без привязки к конкретному объекту.

6.10.2 Чем static-члены отличаются от обычных полей и методов объекта?

Обычные поля и методы связаны с **конкретным объектом**, а static-члены связаны с **классом целиком**. В static-методе нет текущего объекта, поэтому там нельзя использовать `this`, `super` и напрямую обращаться к данным экземпляра.

6.10.3 Когда static уместен, а когда он ломает дизайн?

`static` уместен, когда логика не зависит от состояния конкретного объекта или когда данные действительно общие для всего класса. Он начинает вредить, когда поведение на самом деле относится к объекту, но его делают статическим: тогда теряется нормальная объектная модель, полиморфизм и понятная ответственность объектов. Это уже практический вывод из того, что static-метод не работает с текущим экземпляром.

6.10.4 Что такое final-поле, final-метод и final-класс?

final-поле можно присвоить только один раз. **final-метод** нельзя переопределить в дочернем классе. **final-класс** нельзя наследовать.

6.10.5 Зачем запрещать переопределение метода через final?

Это делают, когда поведение метода должно остаться **строго фиксированным** и не должно меняться в наследниках. Так класс лучше защищает важную логику и свои инварианты.

6.10.6 Что такое nested class и inner class на базовом уровне?

nested class - это любой класс, объявленный **внутри другого класса**. Они бывают двух видов: **static nested class** и **inner class**; при этом **inner class** - это именно **нестатический** вложенный класс, который связан с экземпляром внешнего класса и может обращаться к его членам, даже `private`.

6.10.7 Что такое enum и почему его тоже можно рассматривать как часть ООП-модели?

enum - это специальный тип данных для набора заранее известных констант. Но в Java **enum** - это не просто "список значений": по сути это **полноценный специальный класс**, в котором могут быть поля, методы, конструкторы и даже реализация интерфейсов, поэтому он тоже хорошо вписывается в объектную модель.

6.11 Связи между объектами

6.11.1 Какие отношения между классами бывают кроме наследования?

Кроме наследования, между классами часто рассматривают **ассоциацию**, **агрегацию** и **композицию**. Это уже не ключевые слова Java, а **моделирование отношений** между объектами: кто на кого ссылается, кто кем владеет и насколько сильно связаны их жизненные циклы.

6.11.2 Что такое ассоциация?

Ассоциация - это обычная связь между объектами, когда один объект знает о другом или использует его. При этом объекты могут существовать **независимо** друг от друга.

6.11.3 Что такое агрегация?

Агрегация - это более слабая разновидность отношения "целое-часть", когда один объект содержит ссылки на другие объекты, но эти части обычно могут существовать и отдельно. В базовом Java-коде агрегация обычно выглядит как обычное поле-ссылка; отличие здесь в первую очередь **смысловое и моделирующее**, а не в отдельном ключевом слове языка. Это вывод из UML-модели связей и противопоставления обычной ассоциации более сильной композиции.

6.11.4 Что такое композиция?

Композиция - это сильная связь "целое-часть", при которой часть логически входит в состав целого и обычно **не существует отдельно** от него. Если уничтожается целое, его составные части тоже обычно перестают существовать как самостоятельная часть модели.

6.11.5 Чем композиция отличается от наследования?

При **наследовании** отношение выглядит как "**является**": `Dog is-a Animal`. При **композиции** отношение выглядит как "**имеет**": `Car has-a Engine`. То есть наследование описывает разновидность типа, а композиция - включение одного объекта в другой.

6.11.6 Почему во многих случаях композиция лучше наследования?

Композиция часто лучше, потому что она делает связь **менее жёсткой**. Объекты просто сотрудничают друг с другом, а не зависят слишком сильно от внутреннего устройства родителя, как это часто бывает при неудачном наследовании. Это практический вывод из того, что наследование создаёт отношение специализированного типа, а композиция - отношение включения и использования.

6.11.7 Что означает принцип “has - a” и чем он отличается от “is - a”?

is - a означает “является разновидностью”, и это обычно про **наследование** или реализацию общего типа. **has - a** означает “имеет в составе” или “содержит ссылку на”, и это обычно про **ассоциацию, агрегацию или композицию**. Это один из самых удобных способов на базовом уровне понять, какой тип связи между классами выбран.

6.12 Object и базовые методы объектов

6.12.1 Что такое класс Object и почему он важен?

Класс **Object** - это корневой класс в Java. Это значит, что **любой класс в Java прямо или косвенно наследуется от Object**, поэтому его методы доступны всем объектам. Он важен потому, что задаёт общий фундамент для всей объектной модели языка.

6.12.2 Какие основные методы приходят от Object?

От Object приходят базовые методы, с которыми чаще всего работают почти в любом проекте: **toString()**, **equals()**, **hashCode()** и **getClass()**. Также в Object определены методы, связанные с клонированием и синхронизацией, например **clone()**, **wait()**, **notify()** и **notifyAll()**.

6.12.3 Что делает toString() и зачем его переопределять?

Метод **toString()** возвращает **строковое представление объекта**. По умолчанию оно обычно слишком техническое и плохо подходит для чтения человеком, поэтому метод часто переопределяют, чтобы объект было удобно выводить в логах, отладке и сообщениях.

6.12.4 Что делает equals() и почему его нельзя путать с ==?

Метод **equals()** сравнивает объекты на равенство. В версии из Object он фактически сравнивает **ссылки на один и тот же объект**, а оператор **==** для ссылочных типов тоже проверяет, указывают ли ссылки на один и тот же объект; поэтому **equals()** и **==** совпадают только до тех пор, пока **equals()** не переопределён. Когда **equals()** переопределяют, он обычно начинает сравнивать **смысловое равенство**, например по полям объекта.

6.12.5 Что делает hashCode() и почему он связан с equals()?

Метод **hashCode()** возвращает целое число, которое используется, в том числе, в **хеш-таблицах**. Его связь с **equals()** очень важна: если два объекта считаются равными по **equals()**, то они **обязаны** возвращать одинаковый **hashCode()**.

6.12.6 Почему неправильная реализация equals()/hashCode() ломает коллекции?

Потому что хеш-коллекции вроде **HashSet**, **HashMap** и **Hashtable** опираются на **оба метода сразу**. Если **equals()** и **hashCode()** реализованы несогласованно, объект может, например, успешно добавиться в коллекцию, но потом не находиться, дублироваться или вести себя непредсказуемо при поиске и удалении.

6.12.7 Что делает getClass()?

Метод `getClass()` возвращает объект `Class`, который описывает **реальный класс объекта во время выполнения программы**. Проще говоря, через него можно узнать, к какому именно классу относится данный объект.

6.13 SOLID

6.13.1 Что такое SOLID и зачем эти принципы нужны?

SOLID - это набор из **пяти принципов проектирования объектно-ориентированного кода**: SRP, OCP, LSP, ISP и DIP. Они нужны затем, чтобы код было проще **менять, расширять, тестировать и поддерживать**, не превращая систему в хрупкий набор тесно связанных классов.

6.13.2 Что такое принцип единственной ответственности (SRP)?

SRP означает, что у программного модуля или класса должна быть **одна причина для изменения**. На практике это значит: класс должен отвечать за **одну осмысленную задачу**, а не за всё сразу.

6.13.3 Как понять, что у класса слишком много ответственности?

Если класс меняется по **разным причинам**, если он одновременно хранит данные, считает бизнес-логику, ходит в базу, форматирует вывод и отправляет сообщения - это уже тревожный сигнал. Обычно такой класс имеет **низкую связность внутри себя** и создаёт лишние зависимости снаружи.

6.13.4 Что такое принцип открытости/закрытости (OCP)?

OCP означает, что модуль должен быть **открыт для расширения**, но **закрит для изменения**. Идея в том, что новое поведение лучше добавлять через новые классы, реализации и расширения, а не постоянно переписывать уже работающий код.

6.13.5 Как расширять поведение класса без изменения уже работающего кода?

Обычно это делают через **абстракции, интерфейсы, полиморфизм и композицию**: добавляют новую реализацию, а не переписывают старую. Такой подход уменьшает риск сломать уже существующую логику и лучше соответствует OCP и DIP.

6.13.6 Что такое принцип подстановки Барбары Лисков (LSP)?

LSP означает, что объект подтипа должен быть **корректной заменой** объекта базового типа. Иначе говоря, код, который работает с родительским типом, не должен ломаться, если ему подставили объект дочернего класса.

6.13.7 Почему не любое наследование является корректным с точки зрения LSP?

Потому что внешне класс может наследоваться правильно, а по поведению - нет. Если дочерний класс меняет смысл метода, ломает ожидания клиента, усиливает ограничения или нарушает инварианты базового типа, то такое наследование уже формально есть, но **подстановка работает неправильно**.

6.13.8 Что такое принцип разделения интерфейсов (ISP)?

ISP говорит, что клиентский код **не должен зависеть от методов, которые ему не нужны**. Поэтому вместо одного большого “жирного” интерфейса обычно лучше несколько более маленьких и точных.

6.13.9 Почему лучше несколько маленьких интерфейсов, чем один “жирный”?

Потому что маленькие интерфейсы уменьшают лишние зависимости и делают код точнее по смыслу. Когда интерфейс слишком большой, классы начинают реализовывать методы “для галочки”, а любые изменения в таком интерфейсе задевают слишком много клиентов.

6.13.10 Что такое принцип инверсии зависимостей (DIP)?

DIP означает, что **высокоуровневые** и **низкоуровневые** модули должны зависеть не друг от друга напрямую, а от **абстракций**. Тогда бизнес-логика меньше привязана к конкретным деталям вроде базы данных, HTTP-клиента или внешнего сервиса.

6.13.11 Почему код лучше связывать с абстракциями, а не с конкретными реализациями?

Потому что так снижается связность, и реализации можно легче заменять, тестировать и развивать. Когда код знает только контракт, а не конкретный класс, система становится гибче и лучше переносит изменения.

6.13.12 Как SOLID помогает писать код, который проще тестировать, менять и расширять?

SOLID ведёт к **более высокой связности внутри классов** и **более слабой связности между ними**. За счёт этого классы легче изолировать в тестах, изменения меньше “расползаются” по проекту, а новое поведение проще добавлять через расширение, а не через постоянную переделку старого кода.

6.14 Частые ошибки и плохие практики в ООП

6.14.1 Почему наследование ради переиспользования кода может быть плохой идеей?

Потому что наследование создаёт **жёсткую связь** между классами. Если дочерний класс наследуется только ради готовых методов, а не потому что он действительно является корректным подтипом родителя, такая иерархия становится хрупкой и легко нарушает LSP.

6.14.2 Почему слишком большие классы и “божественные объекты” - это проблема?

Потому что они **знают слишком много и делают слишком много**. Такой класс становится центром зависимостей, его тяжело понимать, безопасно менять и тестировать, а ошибки в нём часто затрагивают сразу много частей системы.

6.14.3 Почему большое количество сеттеров часто делает модель слабой?

Потому что объект перестаёт контролировать собственное состояние и превращается почти в “мешок с данными”. Когда почти любое поле можно в любой момент поменять снаружи, классу сложнее защищать свои правила и инварианты.

6.14.4 Почему static везде подряд - это тревожный сигнал?

Потому что static-члены принадлежат **классу**, а не объекту, и работают **без текущего экземпляра**. На практике это часто признак того, что логика уходит из объектов в процедурный стиль, из-за чего теряются преимущества нормальной объектной модели: состояние объекта, полиморфизм и ясное распределение ответственности.

6.14.5 Почему нарушение инкапсуляции усложняет поддержку кода?

Потому что внешний код начинает зависеть от внутренних деталей объекта. В итоге любое изменение внутри класса рискует поломать код снаружи, а сам объект уже не может нормально контролировать корректность своего состояния.

6.14.6 Почему глубокая иерархия наследования затрудняет понимание программы?

Потому что поведение оказывается размазано по длинной цепочке родителей и потомков. Чем глубже иерархия, тем труднее понять, где именно определён метод, что реально вызывается и не ломает ли один уровень ожидания другого.

6.15 Продвинутые и полезные вопросы по ООП

6.15.1 В чем разница между “объектом как сущностью” и “объектом как структурой данных”?

Если очень просто, **объект как сущность** скрывает данные и показывает поведение, а **структура данных** наоборот - делает данные явными, а операции над ними обычно находятся снаружи. Это два разных стиля моделирования, и у каждого свои сильные стороны.

6.15.2 Почему хорошая ООП-модель должна защищать инварианты объекта?

Потому что объект должен оставаться **корректным** на протяжении всей своей жизни. Если модель не защищает собственные правила, то любой внешний код может перевести объект в неправильное состояние, и тогда остальная система начнёт работать на неверных предположениях.

6.15.3 Что такое слабая связность и высокая связность внутри класса?

Слабая связность между частями системы означает, что компоненты как можно меньше знают друг о друге и меньше зависят от чужих деталей. **Высокая связность внутри класса** означает, что его методы и поля действительно работают вокруг **одной общей задачи**, а не собраны случайно.

6.15.4 Почему композиция часто лучше наследования в реальном проекте?

Потому что композиция обычно даёт **более гибкую и менее жёсткую связь**. При ней объект использует другие объекты как части своего поведения, а не привязывается намертво к глубокой иерархии, где ошибка в выборе базового класса потом долго мешает развивать систему.

6.15.5 Когда абстракция действительно полезна, а когда это лишний слой?

Абстракция полезна, когда она отделяет важный контракт от деталей реализации и реально помогает менять систему без массовых переделок. Но если абстракция создаётся “на всякий случай” и не отражает никакой реальной вариативности, она превращается просто в лишний слой, который усложняет чтение и навигацию по коду.

6.15.6 Как понять, что интерфейс выделен правильно?

Хороший интерфейс обычно описывает **ясную роль** и нужен реальным клиентам, а не придуман “для красоты”. Если интерфейс маленький, понятный, не заставляет реализовывать лишние методы и действительно отделяет контракт от реализаций, значит он, скорее всего, выделен удачно.

6.15.7 Когда полиморфизм делает код чище, а когда только прячет простую логику?

Полиморфизм делает код чище, когда в системе есть **реально разные варианты поведения** и их нужно безопасно расширять. Но если различий почти нет и всё можно выразить одной простой веткой логики без дублирования и жёстких зависимостей, чрезмерная иерархия может только скрыть простую вещь за лишними классами.

6.15.8 Как ООП связано с тестируемостью и расширяемостью системы?

Хорошее ООП помогает выделять ответственности, уменьшать зависимость от деталей и строить код вокруг абстракций. За счёт этого части системы проще тестировать отдельно, легче подменять реализации и безопаснее расширять поведение без переписывания уже работающих модулей.